

Supervised Learning - Report

Kormákur Snorrason

September 2026

1 Introduction

The sport of mixed-martial arts (MMA) is widely regarded as one of the most unpredictable- and dynamic sports one can spectate. A sudden knockout, freak injury or a mistake giving way to a submission, it's a sport where anything, at any moment, can happen. When each bout is equally probable to be a controlled, one-sided domination as it has being an explosive war of violence and unpredictability, the question becomes: can an outcome of such an event even be predicted? Let alone consistently?

The UFC (Ultimate Fighting Championship) is currently the largest professional MMA organization with an extensive record of fights spanning across its 30+ years of existence. With such a long and lustrous history, finding a suitable dataset was not a difficult task and Kaggle, UFC Fight Data - 2026 provided just that.

With records dating back to March 2010 all the way to May of this year, it covers over 7000 fights and includes 57 features that describe each fighters stats., physical attributes and betting odds, and more. Here our target label is **red_winner** (whether the fighter in the red corner won or lost), and from the testing set the red fighter has a win rate of 55.6% against blue's 44.4%; a significant gap.

With this benchmark, the main goal of this project then becomes two-fold:

- Develop a model that can outperform the simple majority class baselines and accurately predict a UFC fight based purely on statistical differences between the fighters.
- See which attributes/features contribute the most to a outcome and by how much of a margin.

2 Process

After having picked a candidate dataset, we needed to verify its quality and usability for a potential model. For this part of the process we conducted some simple, but effective EDA that would answer those questions.

2.1 Brief EDA

We first checked for any obvious outliers and missing data values by skimming over its metadata on Kaggle, of which there were few and far between. For the next step, we moved to analysing the datasets features and it came to us as a pleasant surprise that the dataset didn't just include each fighters raw statistics, but rather the difference between those values at the point before a given bout took place, e.g., `sig_str_diff` (the difference between the red fighters and blue fighters total significant strikes). With the goal of the model being to predict a winner based on comparisons between such attributes, having them pre-calculated lightened the work load significantly. Before committing to this particular dataset, we decided to look into other projects/research material that was conducted using the same dataset. One in particular struck as particularly interesting, that is: Can We Predict UFC Fight Outcomes?. Not only did the results of this article all but confirm to us our initial questions regarding the value of the dataset, it also gave us a good way to validate our own work against theirs. More importantly however, is that it gave us a clear ceiling for the accuracy that any model can achieve and why. There are two cases that need to be examined:

Case 1: A model that is only trained and tested on a dataset that includes a fighters individual statistics (age, significant strikes difference, etc.) is capped around the low-mid 60s for accurately predicting an outcome; a remarkably high number for such an unpredictable sport.

Case 2: Including odds (`red_odds`, `blue_odds`, `odds_diff`) as a feature of the model significantly improves its accuracy (70%), however since it's a feature that exists only as a product of external assessment on a fighters favorability in a given matchup, it collides with the stated purpose of the project.

This becomes more relevant later on, but for now its good to have in mind as we proceed.

2.2 Data Cleaning

After having conducted some initial pruning into the dataset and verified its usability we proceeded to a more extensive data- cleaning and analysis for feature selection.

2.2.1 Outliers & Missing Data

The Return Columns

The values of the `red\_return` and `blue\_return` columns reflect the payout a \$1 bet would return. Including them would cause target leakage since they tell the model exactly who won (loser's return being 0) and in turn lead to a 100% model prediction rate accuracy; something we obviously wanted to avoid.

Stance Outliers

We noticed in the `red\_stance` and `blue\_stance` columns that there were a few rows with faulty classifications and missing data ("Switch " with a trailing space and "Unknown"), that needed to be excluded. We also found one other classification called "Open Stance", which describes an opposite stance matchup, and since we already have the three main stances present ("Orthodox", "Southpaw" and "Switch") it becomes a redundant categorical value and only creates conflict.

Weight Class Outliers

In the column `weight\_class` there we found the value "Catch Weight" which describes fights where one or both fighters missed their target weight and therefore fought outside one of the 12 regulatory weight classes. Since these fights can't be mapped to the standard list of weight classes, we decided to exclude them.

Outliers in Difference Columns

In the `rounds\_diff` column we found one extreme outlier where the value was over 440. We excluded this due to it being an impossible data value when looking at either fighters records (i.e. total fights).

In the `reach\_diff` column there is a single outlier where the reach difference was over 186 caused by one of the fighters reach being measured 0 cm. This is an obvious error caused by a missing data value so we naturally excluded it.

2.2.2 Classifications

We decided to break down the dataset into the following classes:

Gender: MALE — FEMALE

Weight Classes:

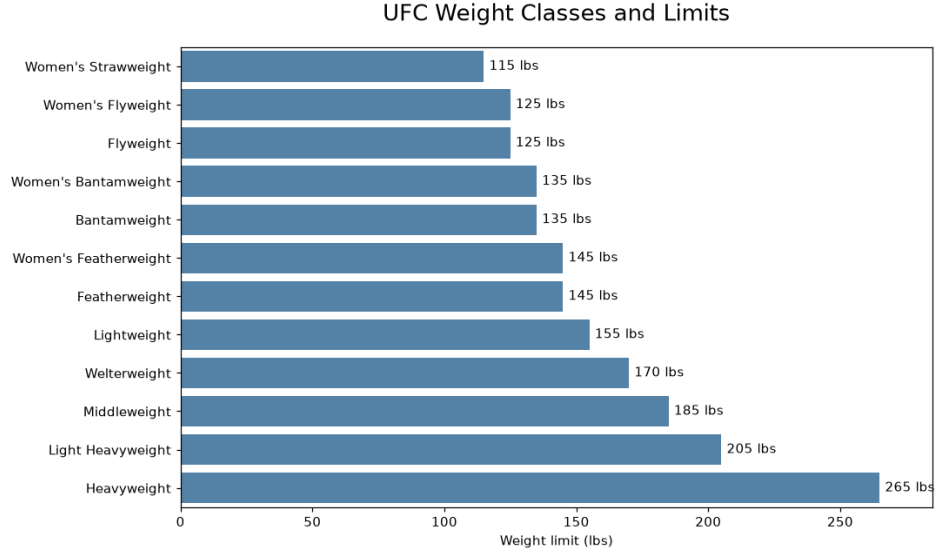


Figure 1: UFC Weight Classes

Stance:

- Orthodox: Left foot forward, favored by right-handed fighters.
- Southpaw: Right foot forward, favored by left-handed fighters.
- Switch: Both Orthodox and Southpaw.

We initially included fighters age as a classifier, whose values were pooled into buckets of age ranges (e.g., 18 - 23) but a realization quickly arose. That is, as we increased the number of buckets (age ranges) the accuracy of the model increased in parallel, which suggest that there is too much variability in a fighters standing in even as little as a year. And since the dataset already included the age difference between fighters for each bout, which is the effect we're interested in, we decided to drop it entirely.

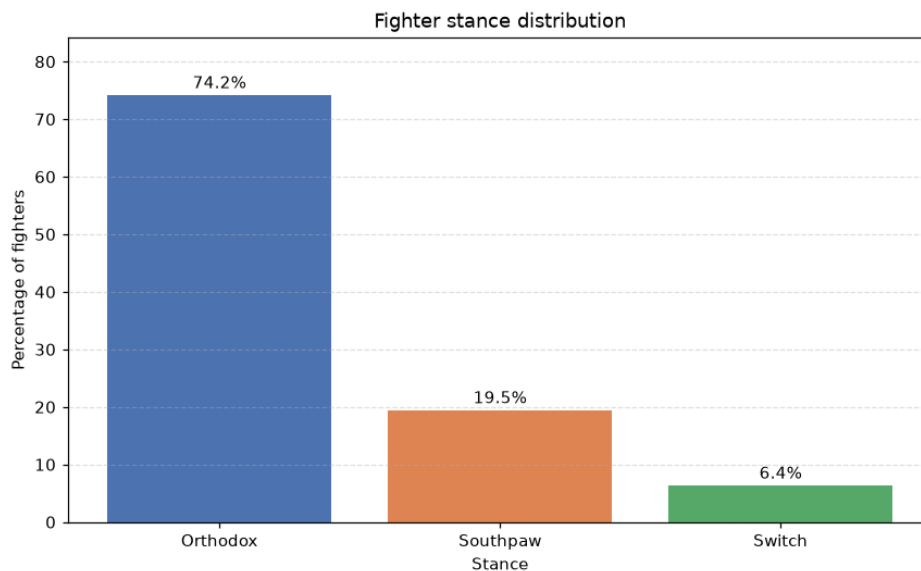


Figure 2: Fighters Stance Distribution

2.2.3 Feature Selection

Along with the aforementioned classifiers, we settled on including only the `\_diff` features due to them providing the model the exact metrics needed to evaluate a fighters standing when making a prediction. The one exception to this was the `odds\_diff` column, and like stated before, the issue with that feature is that it signals to the model which fighter was favored going into a given matchup; biasing it to one outcome based only on external assessment rather than its own.

But although that was the case, it still intrigued us to see what a model with that feature included would deliver. We therefore developed two sets of selected features, one with and one without the `odds\_diff` feature; so we can see first hand how much it impacts the models prediction.

2.2.4 Feature Engineering

The impact of the age difference between two fighters varies significantly depending on where said difference occurs on the age-axis. In a bout between a 25 year old and a 28 year old, the age difference is not nearly as significant as one between a 32 year old and a 35 year old; and the data shows this. In

title fights, fighters 35 and older have a collective win-loss record of 50-75 across all weight classes:

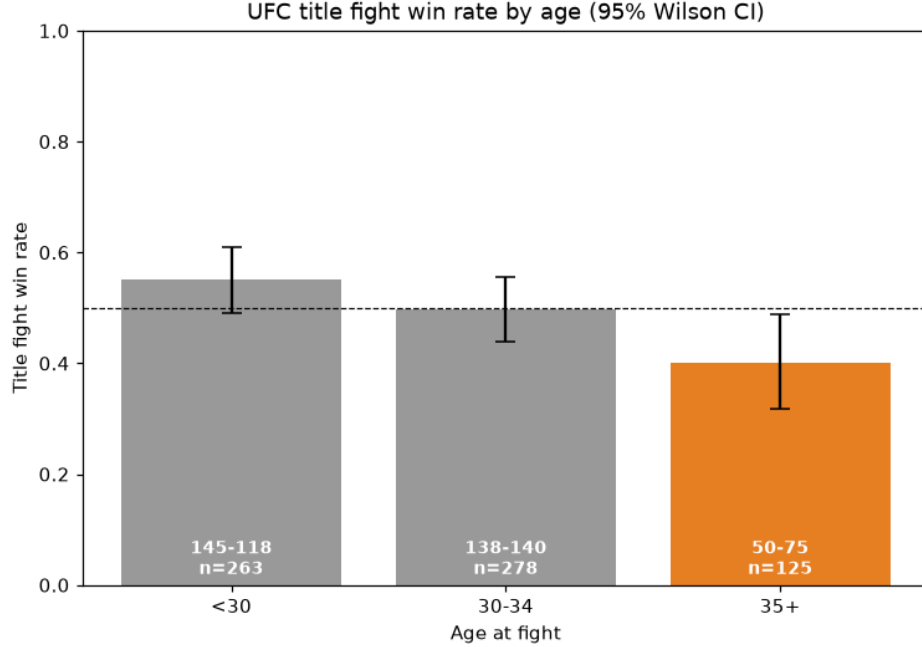


Figure 3: UFC title fight win rate by age (all weight classes)

The distribution is however very skewed, with the heavier weight classes (Middleweight, Light Heavyweight and Heavyweight) boasting a much better higher win-to-loss ratio than the lighter classes; showing that the lighter classes are a lot less forgiving:

Instead of including just the raw age difference, which doesn't take this disparity into account, we created a new feature called `age_diff_x_avg` that numerically encodes how much `age_diff`'s effect should be amplified or dampened based on how old the pair is on average.

2.2.5 Data Split

When it came to splitting the data into dedicated training- and test sets, two questions arose: on which variable does the splitting occur and by how much. We ultimately landed on `fight_date` as our splitting variable due to it providing us two complete datasets that include every selected feature, and selecting June 1st, 2023 as our splitting point gave us clean 80/20 split.

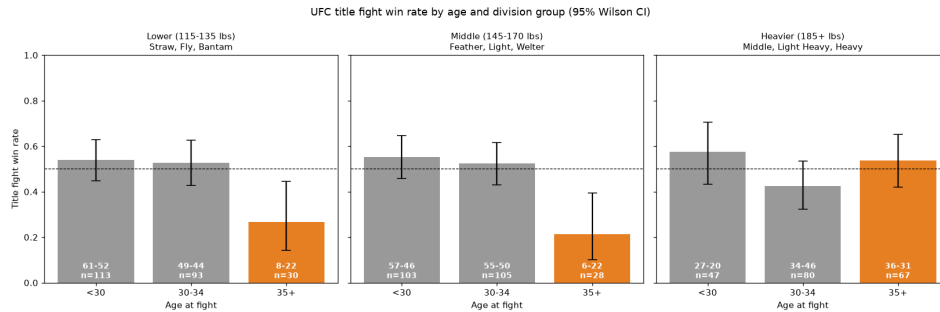


Figure 4: UFC title fight win rate by age & division

Finding the right algorithm's for our models

Logistic Regression

Why did we choose logistic regression as our baseline for accuracy?.

Pros:

- Since our data is split between `\_diff` features and other features, and the `\_diff` features are numeric, that is what logistic regression is built for. So for example, each unit of reach advantage shifts the odds of red winning by some amount.
- Another reason for using logistic regression is it's fast to train and does not take a lot of setup.

Cons:

- A limitation of logistic regression is that it assumes a relatively simple relationship between the features and the log-odds of the outcome. It also does not automatically capture interactions between features. or example, maybe reach advantage has different effects based on what weight class the fight is in.

Why include it: To establish a baseline for comparison, we will be comparing them to a simple logistic regression model.

Random Forest

The reasons we chose random forest for one of our models are.

- Handles the mix of numeric `\_diff` and categorical features without needing scaling, and works well with on-hot encoded categorical features.

- A major benefit of random forests is that they can capture non-linear relationships and interactions automatically, for example, the take-down advantage matters a lot more when the weight difference between the fighters is larger.

XGBoost

The reasons we chose XGBoost for one of our models are.

Pros:

- Like random forests, XGBoost can capture non-linear relationships and interactions. However, instead of averaging many independently trained trees, XGBoost builds trees sequentially, with each new tree attempting to correct errors made by the previous trees. Since our data set is not too big the training time for the models is not too long.
- Captures non-linear relationships and interactions like random forest, but often extracts more predictive power from the same features because each new tree learns from the previous ones.

Cons:

- There are a lot of hyperparameters to tune, and if you don't tune them well the model can be prone to overfitting.
- Requires one-hot encoding for categorical features, which can cause the numbers of features to blow up quite quickly.

Why include it: XGBoost provides a useful comparison with random forest because both can capture non-linear relationships and interactions, but they use different ensemble approaches.

CatBoost

The reasons we chose CatBoost for one of our models are.

Pros:

- Handles categorical features natively, instead of one-hot encoding, this is very useful since we have a lot of categorical features.
- CatBoost can make use of categorical features directly and can capture relationships between categorical variables, such as `red\_stance` and `blue\_stance`. This has a big impact on real fight analysis. For example, the interaction between `red\_stance` and `blue\_stance` could potentially be relevant to fight outcomes.

Cons:

- Is quite similar to XGBoost. It can be slower to train than some of the other models, particularly when using more complex settings.

Why include it: this gives a good way to test whether the native category handling matters, and a good way to find that out is compare XGBoost vs CatBoost.

3 Results

Report on the performance for the different models and hyperparameters. Essentially, take the numbers you got as output of your program and put them into a nice form (plots, tables, ...) that makes them easier to interpret and shows, for example, how the choice of values for different hyperparameters influences the performance.

4 Conclusion

Interpret the results and compare with results in the literature (if possible). What can be said about the performance of different models on the problem? How important is the choice of the hyperparameters for the performance of the different models?

5 Future Work

Suggest how your results could be improved. What did you learn from the results you got and what would you do differently now that you have learned this?